A Grammar of Machine Learning Workflows

Rejecting Data Leakage at Call Time

Simon Roth

April 3, 2026

Abstract

Data leakage has been identified in 648 published papers (as of mid-2024) across 30 scientific fields (Kapoor and Narayanan 2025). The knowledge to prevent it has existed for over a decade; the problem persists because the tools do not enforce what the textbooks teach. No existing ML framework distinguishes formative evaluation (iterative model development) from summative assessment (terminal performance judgment) as a type-level distinction, and none rejects the violation at call time.

This paper presents a grammar (eight typed primitives connected by a directed acyclic graph with four hard constraints) that makes the most damaging leakage types structurally unrepresentable. The core mechanism is a terminal assessment gate: the first call-time-enforced evaluate/assess boundary in an ML framework (as of April 2026), backed by a specification precise enough for independent reimplementation. Boundary-aware splitting addresses the structural layer; the type system catches implementation errors — from human practitioners, LLM code generators, and AutoML systems alike — that would otherwise pass silently through any correct-looking workflow. A companion study across 2,047 datasets (Roth 2026b) grounds the constraints in measured effect sizes. Three directional predictions recorded before observing results: two confirmed, one falsified. Two reference implementations (Python, R) are available.

1 Introduction

Six hundred forty-eight published papers contain data leakage errors across 30 scientific fields (Kapoor and Narayanan 2025). The knowledge to prevent the most common of these errors has existed for over a decade (Kaufman et al. 2012; Cawley and Talbot 2010). The gap is not knowledge. It is enforcement.

Documentation does not close structural failures. Best-practice guides did not prevent an estimated 17.5% of neuroimaging studies from leaking (Rosenblatt et al. 2024). Checklists did not prevent leaked results from propagating into field-level meta-analytic conclusions (van de Mortel and van Wingen 2025). The bias literature arrived at the same diagnosis from a different direction: pipeline stages that should be isolated are not (Roth 2022). Leakage and bias share a root cause — and neither yields to appeals for caution. This paper addresses leakage specifically; fairness bias is a distinct problem addressed elsewhere (Roth 2022).

The field responded with documentation: best-practice guides, checklists (Kapoor et al. 2024), linting tools. Yang et al. (2022)'s LeakageDetector catches three leakage types at 93% accuracy. Drobnjaković, Subotić, and Urban (2024)'s abstract interpretation tracks partition labels through notebook code at 93% precision — the closest existing work to a formal treatment of pipeline correctness. Both detect leakage. Neither prevents it. Detection finds the error after the fact; prevention makes it inexpressible. The question the literature has not answered is whether leakage can be made structurally inexpressible — rejected by the composition rules of the workflow itself, not flagged afterward by a separate tool.

The code generation gap. Large language models now generate ML pipelines faster than any human can review them. Ask one to build a classifier and it will evaluate on the test set, because that is what most training examples do. The leaky path is the short path, and the short path is what language models learn. Preliminary results from a companion study in preparation (Roth 2026a) suggest that the default code path LLMs produce is the leaky one, and asking nicely does not fix it. When pipelines are generated at machine speed, the structural guarantee is the only guarantee that remains.

Structural rejection has worked before. Codd (1970) enumerated eight relational operators and a rejection criterion: queries that violate the relational model are invalid, not merely inadvisable. SQL implemented this. Wickham (2010) did the same for statistical graphics: ggplot2 exposes a curated subset of Wilkinson (1999)'s primitives, and incomplete specifications fail at construction. Both grammars survived because they had executable implementations. The primitives of the supervised ML lifecycle are already standardized (sklearn established fit/predict/transform; tidymodels added per-fold preprocessing). What is missing is the Wickham move: a curated selection with a rejection criterion that makes "this workflow is structurally invalid" a type error rather than an opinion. When pipelines are generated at machine speed — by AutoML sys-

tems, LLM code generators, and automated retraining schedules — structural enforcement is the only guarantee that scales regardless of who or what writes the code.

The Wickham gap. Wickham (2010)'s contribution was not the API; it was the selection. ggplot2 is not a complete implementation of Wilkinson's full system; it exposes a curated subset of it. By choosing which primitives to make primary and which to omit, Wickham shifted which errors were easy to make: incomplete specifications fail at construction, and correct compositions become the default path. The analogous selection has not been made for the ML lifecycle. Kuhn and Silge (2022) came closest: recipes inside tidymodels workflows enforce per-fold preprocessing structurally, and `last_fit()` encourages a one-shot terminal evaluation. The tidymodels framework got most of the way there. But a user can still fit a recipe outside a workflow, evaluate on training data, or reuse the test set. The structure discourages mistakes but does not eliminate them. There is no type that distinguishes "this data has been split correctly" from "this data has not." The primitives are typed at the implementation level (recipe, parsnip model, workflow), not at the correctness level (Partition, PreparedData, Model).

This paper makes that selection for supervised learning, addressing the question with a *grammar* in the Wilkinson–Wickham sense: a compositional system with typed primitives and a rejection criterion for a domain, not a generative rule system in Chomsky's sense (the distinction is elaborated in Section 4). The underlying mechanism is analogous to information flow control (Myers 1999): train, valid, and test function as security levels, and the guards enforce that test-labeled data cannot flow into training operations at primitive boundaries (Section 4). Specifically, it addresses tabular supervised learning on the four leakage classes identified in the companion study (Roth 2026b). Temporal leakage is addressed by boundary-aware primitives (`split_temporal`, `cv_temporal`); feature engineering is scoped per fold by `prepare` after `split`, and `fit` rejects unregistered data that bypassed `split` entirely; distribution mismatch across groups is addressed by `split_group`. What the grammar cannot see is contamination baked into the raw data before any primitive touches it — a feature that encodes the target under a different name, or a generating process that shifted between collection and deployment.

The specification (Appendix A) is the paper's core artifact. It defines the type system, primitive operations, guards, and validity conditions precisely enough for independent reimplementation. The conformance conditions are stated in both structured English and predicate notation; what it lacks is inference rules and a soundness proof, but it is complete enough that conformance is testable from the API surface. Read the Appendix as the specification; read the rest as the argument for why it should exist.

2 A Motivating Example

A student who grades their own homework learns nothing new. This code grades its own homework:

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import roc_auc_score

X_train, X_test, y_train, y_test = train_test_split(X, y)

for Model in [LogisticRegression, RandomForestClassifier]:
    model = Model().fit(X_train, y_train)
    p = model.predict_proba(X_test)[:, 1]
    print(f"{Model.__name__}: {roc_auc_score(y_test, p):.3f}")
```

Each model's test-set score is computed and compared. The researcher picks the model with the higher test AUC and reports it. This is methodologically incorrect: the test set has been used for model selection, not just final assessment. The measured inflation is small at $K = 2$ but grows logarithmically with K: at $K = 10$ the effect is $d_z = 0.929$, at $K = 19$ it is $d_z = 0.94$ (Roth 2026b). No sklearn (Pedregosa et al. 2011) mechanism prevents this: `train_test_split` returns ordinary arrays; nothing stops the user from scoring on `X_test` repeatedly and choosing the best result. The textbook rule ("use the test set only once") is documentation, not enforcement.

The grammar version:

```
import ml

s = ml.split(df, target="y", seed=42)

for algo in ["logistic", "rf"]:
    model = ml.fit(s.train, "y", algorithm=algo, seed=42)
    metrics = ml.evaluate(model, s.valid)
    print(f"{algo}: {metrics['roc_auc']:.3f}")

# pick winner, fit on full dev, assess once
best = ml.fit(s.dev, "y", algorithm="rf", seed=42)
final = ml.assess(best, test=s.test)
```

Each line produces a typed output that constrains what the next line can accept.

`split` creates a Partition (train, valid, test, and dev) with provenance tracked by content hash. `fit` accepts only data registered by `split`; anything else is rejected before training begins. `evaluate` checks the model against validation

data and returns Metrics: repeatable, cheap, safe for iteration. This is the *iterate zone*: the practitioner re-fits and evaluates as many times as needed.

Only the final model proceeds to `assess`. It is fit on `s.dev` (train + valid combined) to use all available non-test data. `assess` returns Evidence: terminal, sealed, irreversible. A second call on the same holdout fails the guard. The methodologically incorrect pattern in the sklearn version is now structurally rejected: the first `assess` marks the holdout as spent in the provenance registry, and no amount of cloning, serializing, or restarting recovers it within the session.

The canonical sklearn leakage example, normalizing before splitting (Class I), is structurally visible but empirically negligible ($d_z \approx 0$). The grammar prevents it too (unregistered data is rejected by `fit`), but the core contribution addresses the patterns a `Pipeline` does not: repeated test-set assessment ($d_z = 0.929$, ΔAUC $= +0.040$) and training on evaluation data ($d_z = 0.372$–1.112, ΔAUC $= +0.004$–0.024).

3 The Grammar

3.1 Kernel primitives

Eight operations. That is the entire grammar. Everything a supervised learning workflow does (splitting, cross-validating, preprocessing, fitting, predicting, evaluating, explaining, assessing) reduces to these eight primitives and the types that connect them:

#	Primitive	Type signature	What it does
1	**split**	DataFrame → Partition	Create train/valid/test splits with provenance
2	**cv**	DataFrame[g] → CVResult	Create k-fold rotation from training data
3	**prepare**	DataFrame[g] → PreparedData	Normalize, encode, impute features
4	**fit**	(DataFrame[g] | CVResult) × target → Model	Train a model
5	**predict**	Model × DataFrame → Predictions	Apply model to new data
6	**evaluate**	Model × DataFrame[g] → Metrics	Measure on validation data (repeatable)
7	**explain**	Model [× DataFrame] → Explanation	Feature importances, partial dependence
8	**assess**	Model × DataFrame[g] → Evidence	Measure on test data (terminal, once)

[g] Guarded: requires a DataFrame with split provenance (registered by `split`).

`assess` and `explain` are terminal: their output types (Evidence, Explanation) feed no further primitive. `cv` transforms a training or dev DataFrame into a CVResult, a rotation schedule of k train/validation index pairs — separating boundary creation (`split`) from iteration (`cv`). Cross-validation is a non-test operation: `cv(s.train)` is the common case; `cv(s.dev)` is valid when the practitioner wants to cross-validate over the full development set. The test partition is never valid input. In v1.0, cross-validation was a variant inside Partition; extracting `cv` into its own primitive made the two concerns independently observable and swappable.

`explain` and `predict` carry no partition guard. Every other kernel primitive carries at least one partition guard: `prepare` and `fit` require registered `{train, valid, dev}`-tagged data; `evaluate` rejects test-tagged and unregistered data; `assess` requires test-tagged data and fires once per holdout. `explain` and `predict` require only a fitted Model. Both may be called on any data, any number of times, before or after `assess`. Explanation and prediction are diagnostic, not part of the validity chain; the grammar has no partition guard to place on them.

The **assess-once** constraint is a call-time guard, not a type error: a second `assess` on the same holdout fails at runtime (Section 4). `fit` accepts `{train, valid, dev}`-tagged data and handles preparation internally per fold; calling `prepare` explicitly is available but never required.

The iterative/final `fit` distinction. The type signature `fit :: DataFrame × target → Model` is identical whether `fit` is called during the iterate zone (training a candidate model per fold, evaluated and discarded) or as the final `fit(dev)` call after hyperparameters are fixed. These two roles differ in commitment: per-fold models are expendable; the final `fit(dev)` model is the one that proceeds to `assess`. The grammar assigns both calls the same type because they satisfy the same preconditions. The semantic distinction (which `fit` is final) is invisible to the type system; the diagnostics layer catches it.

Why 8? The count is a design argument, not a theorem: 8 is the smallest set that covers the supervised learning lifecycle without collapsing a type boundary or losing a guard. Five primitives carry the load-bearing guards: `split`, `prepare`, `fit`, `evaluate`, and `assess`. The remaining three (`cv`, `predict`, `explain`) are structurally necessary: `predict` and `evaluate` both consume a Model, but `evaluate` requires `partition_tag != test`; merging them drops a guard. `prepare` returns a reusable Transformer; absorbing it into `fit` destroys consistent preprocessing of held-out data. `cv` produces CVResult; absorbing it into `split` hides rotation inside boundary logic. Removing any primitive loses either a type or a guard.

3.2 The type DAG

The primitives connect through a typed directed acyclic graph (Figure 1). Data flows downward from a single DataFrame through a split decision into three partitions. The `cv` primitive makes the fit/evaluate loop concrete: it transforms the training DataFrame into a rotation schedule (k train/validation index pairs) that `fit` consumes fold by fold. Only after iteration converges does the model proceed to terminal assessment on held-out test data, producing evidence that cannot be revisited.

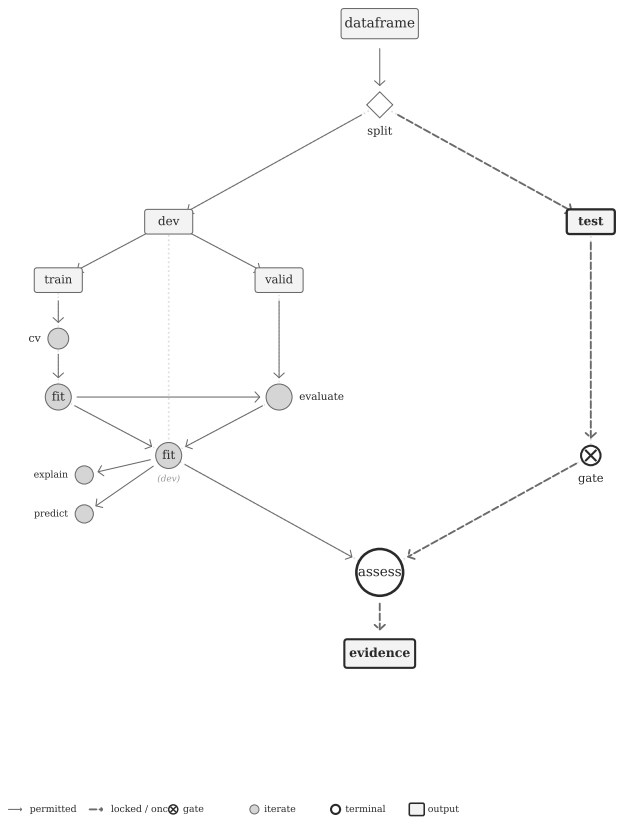

Figure 1: The ML workflow grammar as a typed DAG. Diamond: split decision producing dev and test partitions. `cv` transforms the training DataFrame into train/valid fold rotations. Solid arrows: permitted data flow. Dashed arrows: locked until terminal assessment. Gate (×): test partition held until `assess` is called once, producing terminal evidence. The fit–evaluate loop iterates on train/valid folds; `fit(dev)` retrains on the full development set before the final commitment.

The Partition type exposes `.train`, `.valid`, `.test`, and `.dev` (train ∪ valid). Cross-validation (Stone 1974) is a separate primitive (`cv`) that transforms a training DataFrame into a CVResult with `.folds` and `.k`; the test partition stays on the original Partition and is accessed at `assess` time. The full workflow with cross-validation:

```
s = ml.split(df, target="y", seed=42)
c = ml.cv(s.train, folds=5, seed=42)
model = ml.fit(c, "y", algorithm="rf", seed=42)
metrics = ml.evaluate(model, s.valid)        # iterate zone
best = ml.fit(s.dev, "y", algorithm="rf", seed=42)
evidence = ml.assess(best, test=s.test)      # terminal, once
```

`cv` takes the training DataFrame and creates a k-fold rotation schedule; cross-validation is a training-set operation (Hastie, Tibshirani, and Friedman 2009, Ch. 7); the test partition remains on the originating Partition, untouched. `fit` consumes the CVResult and trains across folds; `evaluate` measures on the held-out validation set (repeatable); `assess` produces sealed Evidence on test data (terminal). A second `assess` on the same holdout fails the guard. Preparation is handled inside `fit` by default (per fold); calling `prepare` explicitly is available for full control:

```
prepped = ml.prepare(s.train, target="y")  # fit on train only
model = ml.fit(prepped, "y", algorithm="rf", seed=42)
```

Domain specializations. The implementation exposes `split`, `split_temporal`, and `split_group` as independent entry points, each with domain-specific parameters and guards. `split_temporal` carries guards that `split` does not: embargo enforcement, expanding-vs-sliding window policy, temporal ordering with no future leakage. `split_group` carries a group-non-overlap guard that is meaningless for random splits. The same pattern applies to `cv`, `cv_temporal`, and `cv_group`. All specializations share the type signature `DataFrame → Partition` (or `DataFrame → CVResult` for cv variants); what differs is the guard system.

The specializations are constraint profiles, not type distinctions: each conjugates the same grammatical verb for a different scientific domain. Time series forecasting, where expanding-window validation prevents future leakage (e.g., Ballarin et al. 2024). Grouped data with repeated observations per unit, where subject-level splitting prevents within-unit leakage (e.g., Tampu, Eklund, and Haj-Hosseini 2022). Standard cross-sectional ML. The primitive count remains 8; the API surface reflects that different fields require different structural constraints on the same operation.

3.3 The four hard constraints

#	Constraint	What it prevents	Leakage class	ΔAUC
1	**Assess once per holdout**	Repeated test-set peeking	Class II (d_z = 0.929)	+0.040
2	**Prepare after split, per fold**	Global preprocessing leakage	Class I ($d_z \approx 0$)	≤ 0.005
3	**Type-safe transitions**	Fitting on test/untagged data; evaluating without a fitted model	Class II/III (d_z = 0.372–1.112)	+0.004–0.024
4	**No unregistered data into `fit`**	Feature selection using test labels	Class II (d_z = 0.929)	+0.040

Constraints 1 and 4 both target Class II leakage; they differ in mechanism (per-holdout registry vs. provenance gate at `fit`). The companion study (Roth 2026b) measured the effects these constraints prevent: at the corpus median ($n \approx$ 1,900), peeking inflates by $d_z = 0.929$ and seed cherry-picking by $d_z = 0.885$ — enough to flip a model from "below threshold" to "deployable." Class III memorization effects (d_z = 0.372–1.112) scale with model capacity. Constraint 2 addresses an effect that is empirically negligible ($d_z \approx 0$). But it costs nothing (per-fold preparation is the default), and no reviewer will object to a grammar that prevents what textbooks unanimously recommend against.

Four rules. Codd needed twelve relational rules and eventually struggled with the count. Chomsky's early programme required increasingly many syntactic rules before principles-and-parameters reduced the apparatus. The grammar chooses minimality: four constraints that cannot be violated, everything else left to the implementation.

3.4 Strategy families

A practitioner does not call `split`, `fit`, `evaluate`, `assess` one at a time. They compose them into workflows: screen several algorithms, tune the winner, stack an ensemble, assess the result. The 8 primitives compose into 4 strategy families that cover this full developmental cycle:

Family	Question	Strategies
Selection	Which algorithm?	screen, compare, pick
Optimization	Which hyperparameters?	tune (grid, random, bayesian)
Evaluation	How to rotate data?	kfold, grouped, temporal, nested_cv
Ensemble	How to combine models?	blend, bag, boost, stack

The strategies decompose into kernel primitives:

- **screen**: ∀ algo: fit → evaluate → rank → Leaderboard
- **tune**: ∀ params: fit → evaluate → pick → TuningResult
- **stack**: ∀ model: fit → out-of-fold predict → fit_meta → StackedModel

No strategy requires a new primitive: each reduces to typed applications of the 8 kernel verbs. That the core developmental workflow fits within this decomposition (for the strategies examined here) is evidence that the primitives capture structure, not an accidental API boundary. The output containers Leaderboard, TuningResult, and StackedModel are informal wrappers for composition results; they are not declared types in the grammar's type system.

The erased intermediate in `tune`. The `tune` decomposition hides a type the grammar does not declare: after `pick` identifies optimal parameters, those parameters — a `HyperParameters` intermediate — feed the final `fit(dev)` call. The grammar's visible type chain is `TuningResult → fit → Model`; the `HyperParameters` intermediate exists in every implementation but is not a sort in the type DAG. The compression is intentional: `tune` is a strategy, not a primitive, and its internal types are outside the grammar's minimal vocabulary. A conforming extension could expose `HyperParameters` as a declared sort with its own typed accessor from `TuningResult`, making the refitting step explicit.

Grammar predictions. The constraints and composition rules generate testable predictions: if a strategy violates a constraint, measurable inflation should follow. Three were tested in the companion study before observing results:

Prediction	Source	Result
Screen inflation ($d_z > 0$): selecting best-of-K algorithms inflates performance	Selection family; consistent with Cawley and Talbot (2010)	Confirmed: $d_z = +0.27$, K-invariant
Stack leakage ($d_z > 0$): out-of-fold meta-learner leaks through fold labels	Ensemble family	**Falsified**: d_z = -0.22; `stack()` is empirically safe
Seed inflation ($d_z > 0$): reporting best-of-S seeds inflates performance	Assess-once constraint	Confirmed: $d_z = +0.89$, prevalence 92%

Two of three confirmed; one falsified. The confirmed predictions were not equally risky: screen inflation was consistent with Cawley and Talbot (2010)'s prior findings on selection bias, so its confirmation was expected. Seed inflation was the more novel claim. The falsified prediction (stack leakage) was structurally safe to lose: the grammar's correctness does not depend on stacking being leaky. The falsification is evidence that the grammar generates specific, wrong-able hypotheses, not post-hoc rationalization. The full experimental results are in Section 7.

The grammar generates further predictions. The companion study already provides partial evidence for some (nested CV bias is measured indirectly through the HP tuning experiment; the neural network capacity prediction is stated in both papers). Two predictions remain genuinely open: (1) a between-subjects experiment should show that the grammar's structural enforcement reduces leakage rates in human-written code compared to sklearn — if it does not, the grammar is correct but practically inert; (2) the grammar should produce the same leakage reduction for LLM-generated code as for human-written code — if the structural guarantee holds, the author's species should not matter.

3.5 The terminal boundary

In 1967, Scriven (1967) drew a line that education research has respected ever since: *formative* evaluation improves the thing being developed; *summative* evaluation judges the finished product. The two serve different purposes and must not contaminate each other. ML has the same distinction and has lost it. In sklearn, `cross_val_score` reports a quantity called `test_score` that is formative feedback, not a terminal test. In tidymodels, the "assessment set" is a held-out CV fold used for iteration, not for judgment. When the same word covers both, nothing in the vocabulary marks the boundary — and 648 papers crossed it.

The grammar recovers the distinction as a type boundary. `evaluate` is the homework — formative, repeatable, safe to learn from. `assess` is the exam — summative, once, irreversible. The type system is the wall between them, enforced at call time.

- **Evaluate** operates on validation data. It is repeatable, cheap, and safe for decisions. It informs the iterate cycle: the practitioner reads the Metrics output, adjusts, and fits again. Metrics is terminal in the type system (no primitive accepts it as input), but it drives iteration at the practitioner level.
- **Assess** operates on test data. It is terminal, once per holdout, and irreversible. It is the commit: the model's performance on data it has never been influenced by or seen.

The terminal boundary prevents accumulation of selection pressure. It does not eliminate per-decision noise: at practical dataset sizes, every selection from K alternatives carries an expected inflation of $\sigma\sqrt{2\ln K}$ that decays as $1/\sqrt{n}$ but does not vanish at typical n (Roth 2026b). The $\sigma\sqrt{2\ln K}$ bound assumes independent evaluations with equal variance — conditions violated by correlated fold evaluations and heterogeneous algorithms. It is a first-order approximation, not an exact decomposition; the companion study validates it empirically ($R^2 > 0.99$ for $\log K$ scaling). The grammar's contribution is making the summative decision the *only* decision allowed to touch the test set within a given research cycle — bounding the total selection pressure to one terminal query rather than letting it accumulate across an uncontrolled sequence of evaluations.

The sample-splitting logic is textbook (Hastie, Tibshirani, and Friedman 2009, chap. 7): hold out a test set for generalization estimation (§7.2), use cross-validation for model selection (§7.10), and never preprocess before splitting (§7.10.2, "the wrong way"). The grammar does not add new statistical insight; it types the boundaries that the textbook explains and that hundreds of papers violated. Cross-validation becomes `cv(s.train, folds=k)` feeding the Evaluation family. The two textbook failure modes (preprocessing before splitting; reusing the test set, §7.7) map to Constraints 4, 2, and 1. Bates, Hastie, and Tibshirani (2024) showed that cross-validation estimates the prediction error of the *procedure* at sample size $n(1 - 1/K)$, not the specific fitted model. This distinction matters for the grammar: the Metrics type produced by `evaluate` measures the procedure's expected performance under resampling, not the deployed model's future accuracy. A practitioner who treats CV scores as model-level guarantees is committing a category error that the grammar's type distinction makes visible — Metrics (procedure-level, formative) versus Evidence (test-partition-level, terminal).

Bates et al.'s analysis operates entirely within the evaluate zone and makes no claim about terminal holdout assessment. The Metrics/Evidence type distinction is this paper's extension of their finding, not their conclusion. The grammar provides the complementary formalization for the terminal boundary: the evaluate/assess distinction, implicit in every textbook's sample-splitting advice, is enforced by the types, not by the student's memory.

4 Design Properties

The grammar makes three promises. Each can be checked by anyone reimplementing the specification. They are design constraints, not theorems, and they hold by construction rather than by proof.

Formal characterization. The system uses call-time guards in the spirit of typestate (Strom and Yemini 1986): what you can do next depends on where you are. The assess-once constraint is genuine typestate (a specific holdout transitions from unassessed to assessed); the broader workflow — where `split` produces a Partition and `fit` produces a distinct Model — is closer to a session protocol than per-object typestate. The term "grammar" is used in the Wilkinson–Wickham sense — a decomposition of a domain into primitives with composition rules and a rejection criterion — not Chomsky's generative sense.

The type system has four states. Each primitive either advances the state, rejects the call, or does not apply. The transition table makes the rejection criterion visible at a glance: every "reject" cell is a leakage vector that the grammar closes.

State	`split`	`cv`	`prep.`	`fit`	`eval.`	`expl.`	`pred.`	`assess`
Untagged	Partit.	rej.	rej.	rej.	rej.	rej.	rej.	rej.
Partitioned	-	CVRes.	PrepData	Fitted	rej.	rej.	rej.	rej.
Fitted	-	-	-	-	Metrics	Explan.	Predict.	Evidence
Assessed	-	-	-	-	Metrics	Explan.	Predict.	rej.

States correspond to the data/model lifecycle: Untagged (raw DataFrame), Partitioned (after `split`), Fitted (after `fit`), Assessed (after `assess`). "reject" means the call fails with an error; a dash in the table means the verb does not apply to that state's type. `explain` and `predict` remain available after assessment; `assess` fires once per holdout (Fitted → Assessed) and rejects any subsequent call on the same test partition.

The grammar says no in two ways. Structural rejection is static: a function that accepts Model cannot receive a Partition — the types are incompatible and the call never begins. Guard rejection is dynamic: the types match but the provenance registry remembers that this test partition has already been assessed, and the guard fires before the call completes. The four hard constraints are four policies enforced by one mechanism (type checking + guard checking), each targeting a distinct leakage vector (three address Class II through different mechanisms; one addresses Class I). Assess-once is a guard; the type DAG prevents nonsensical type chains structurally. The policies are independent (each addresses a specific leakage vector); the enforcement mechanism is shared. Full algebraic formalization of the guard system is future work.

4.1 Connectivity (not algebraic closure)

A grammar with dead-end types forces the practitioner to leave the type system and work around it. This grammar has no dead ends: every type in the DAG is reachable from DataFrame through the primitives. The non-terminal types form three reachable paths from DataFrame: a direct path (DataFrame → Partition → Model, via `fit`), an explicit-prepare path (DataFrame → Partition → PreparedData → Model, via `prepare` then `fit`), and a CV branch (Partition → $\text{DataFrame}_{\text{train}}$ → CVResult → Model, via fold-aware `fit`). The direct path is the default; explicit preparation is optional. The terminal types (Predictions, Metrics, Evidence, Explanation) are all reachable from Model. No type is orphaned; any conforming extension must preserve this connectivity.

The grammar is not closed in Codd's sense. In the relational algebra, every operator returns a relation, so operators compose freely. Here, `predict` returns Predictions, not a DataFrame; you cannot call `split` on a prediction. That is the point: the types that prevent leakage are the same types that prevent free composition. A fully composable grammar would let you pipe anything into anything, including leaky workflows. Free composition is not the design goal; correctness is.

4.2 Generativity

A grammar that limits how many models you can try before committing would be intolerable. This grammar does not: split → (fit → evaluate)^n → assess is valid for any n. Each fit and evaluate call draws from the same Partition produced by split, not from the Metrics output of the previous evaluate (which is terminal). The cycle exists in the workflow pattern, not in the linear type chain. A weak form of generativity; any system with a loop generates an infinite language. The claim is not about expressiveness class (the grammar's formal language-theoretic classification is an open question) but about practical scope: the grammar does not artificially restrict how many models a practitioner trains before committing to assessment.

Scope. The grammar covers batch supervised learning on a complete dataset available at split time, including automatic production retrain cycles. Multi-task learning (multiple simultaneous targets), self-supervised learning (no explicit labels), few-shot learning (N too small to partition meaningfully), federated learning (data never centralized), and online learning (incremental data arrival) each violate at least one foundational assumption and fall outside the grammar's scope. The grammar permits all valid workflows within its scope (no valid workflow is rejected) and makes no claim about valid ML workflows in general.

4.3 Rejection criterion

Here is where the grammar earns its keep. Each row below is a workflow that looks reasonable, runs without error in sklearn, and produces a number that is wrong. The grammar rejects all of them:

Invalid workflow	Why rejected	Leakage class
assess → assess (same holdout)	Terminal: once per holdout	Class II
prepare(all) → split → fit	Prepare must follow split	Class I
select_features(X, y) → split → fit	No unregistered data into `fit`	Class II
evaluate(data) without prior fit	Type error: no Model	Class III
prepare(test)	Guard: test tag not in {train, valid, dev}	Class I
fit(test)	Guard: test tag not in {train, valid, dev}	Class III
evaluate(test)	Guard: test tag reserved for assess	Class II

Concretely, an invalid workflow raises at the verb boundary:

```
s = ml.split(df, target="y", seed=42)
m = ml.fit(s.dev, "y", algorithm="rf", seed=42)
e1 = ml.assess(m, test=s.test)   # succeeds
```

```
e2 = ml.assess(m, test=s.test)   # fails
# PartitionError: test holdout already assessed
```

Assess-once (by construction). Constraint 1 specifies: for test partition t, assess(m, t) is valid if and only if t has not been previously assessed in the provenance registry, regardless of which model. After the first call, t is marked as assessed; any subsequent call on the same holdout — same model or different — fails the guard. The implementations additionally set m.assessed ← true as a fast-path diagnostic, but the per-holdout check is the structurally load-bearing enforcement. Similarly, prepare(all_data) before split violates Constraint 2 (the data is unregistered in the provenance registry, so the guard rejects it), and select_features(X, y) before split violates Constraint 4. All three are rejected at call time. These are not theorems; they are consequences of the type guards. The claim is that any conforming implementation must enforce them. The Appendix (§ Invalid workflows as guard failures) walks through each entry in the table above.

This grammar is a pure rejection function, not a generation function: its value lies not in the workflows it enables but in the workflows it rejects at call time.

The rejection is *positional*, not *structural*. You can write the invalid workflow. You cannot execute it. Chomsky's grammar makes certain strings underivable: the invalid form cannot be generated at all. Codd's constraints make certain states unstorable: the invalid row cannot exist in the relation. The ML grammar occupies a third position — it rejects at the verb boundary: `ml.fit(s.test, "y")` is a syntactically valid Python expression, but it raises `PartitionError` before any computation begins. The invalid workflow is writable in the host language but not executable through the grammar's API. This is the price of embedding a grammar in a dynamically typed host: the grammar cannot prevent you from writing the line, only from running it. Runtime guards at every verb entry point are the strongest enforcement available without a native partition type in the host language.

The grammar's `partition_tag` functions as an information flow label (Myers 1999): train, valid, and test are security levels, and the guards enforce that test-labeled data cannot flow into training operations. The analogy is instructive but limited: in full information flow control, a type system with label lattices tracks implicit flows through branches and joins; the grammar enforces only direct flows at primitive boundaries, which is access control rather than full non-interference. Leakage prevention is information flow control applied to the train/test boundary rather than the public/private boundary. Drobnjaković, Subotić, and Urban (2024)'s abstract interpretation approach arrives at the same insight from the static analysis side; the grammar enforces it dynamically.

All four rules are purely negative: they define the boundary of valid workflows without ranking or recommending within it. The grammar does not specify which algorithm to choose, how many folds to use, or when to stop iterating;

those are semantics-layer decisions. The positive structure of a good ML workflow is the diagnostics and strategy layer's job, not the grammar's.

To my knowledge, no existing ML framework enforces a terminal assess-once constraint at call time, rejecting the violation rather than detecting it afterward. The grammar does not lint completed workflows; it rejects invalid ones as they are constructed. This claim is bounded to the grammar's own type system: operations outside the grammar's eight primitives can always produce leakage that the grammar cannot see.

The workflow's structural guarantees are a minimum over its constituent primitives: a violation at `split()` propagates through all downstream operations, and no downstream primitive recovers correctness from an upstream violation. Hence the four hard constraints are placed at `split` and `prepare` rather than at `assess`: the terminal boundary is too late to undo upstream contamination.

5 Embedding in Code

5.1 Implementation

The grammar is implemented in two languages: Python (`mlw` on PyPI, imported as `import ml`) and R (`ml` on CRAN). Both expose all 8 kernel primitives with identical type signatures:

```python
# Python
import ml

s = ml.split(df, target = "y", seed = 42)
model = ml.fit(s.train, "y", algorithm = "rf", seed = 42)
metrics = ml.evaluate(model, s.valid)
final = ml.assess(model, test=s.test)
```

```r
# R
library(ml)

s <- ml_split(df, target = "y", seed = 42)
model <- ml_fit(s$train, "y", algorithm = "rf", seed = 42)
metrics <- ml_evaluate(model, s$valid)
final <- ml_assess(model, test = s$test)
```

The portability claim rests on two reference implementations (Python and R) converging on the same behavior across language boundaries. The specification in the Appendix is the authoritative reference; any implementation that satisfies the conformance conditions below is conforming regardless of host language.

5.2 Conformance conditions

Codd (1970) defined the relational model as a mathematical specification; the later "Codd rules" formalized a practical conformance standard against it. I propose conformance conditions in the same spirit — but the analogy is limited: Codd's anchor was mathematical (relational completeness); the grammar's is empirical (leakage effect sizes that justify which constraints exist). The eight conditions below test whether an implementation does what the specification says it should, not whether the specification is complete.

An implementation is conforming if and only if it satisfies all eight. Each is testable from the API surface without reading source code:

1. `split` produces a Partition with tagged train/valid/test/dev DataFrames
2. `fit` requires a registered DataFrame with `partition_tag` ∈ `{train, valid, dev}`; unregistered or test-tagged data raises `PartitionError`
3. `evaluate` and `assess` require a fitted Model
4. `assess` requires test-tagged data that has not been previously assessed; after the call, the holdout is marked as assessed and any subsequent call on the same partition fails (regardless of model)
5. In declarative mode, `fit` applies `prepare` independently per fold — the scaler/encoder state from fold k does not leak into fold $k+1$
6. Evidence is a named class not substitutable for Metrics in the host language; `isinstance(result, Evidence)` is distinguishable from `isinstance(result, Metrics)`
7. `cv` produces a CVResult that blocks direct access to `.train`, `.valid`, `.test`, and `.dev`; the test partition remains on the originating Partition
8. `evaluate` rejects test-tagged data; test data is reserved for `assess`

In predicate notation, where $\text{tag}(d)$ denotes the partition tag of DataFrame d, $\text{reg}(d)$ denotes registration in the provenance registry, and $\text{assessed}(t)$ denotes whether test partition t has been assessed:

(1) $\texttt{split}(d) \to \text{Partition}(p)$ where $\text{tag}(p_i) \in \{\text{train}, \text{valid}, \text{test}, \text{dev}\}$

(2) $\texttt{fit}(d) \Rightarrow \text{reg}(d) \wedge \text{tag}(d) \in \{\text{train}, \text{valid}, \text{dev}\}$

(3) $\texttt{evaluate}(m, d) \Rightarrow \text{fitted}(m); \quad \texttt{assess}(m, t) \Rightarrow \text{fitted}(m)$

(4) $\texttt{assess}(m, t) \Rightarrow \text{tag}(t) = \text{test} \wedge \neg\text{assessed}(t); \;\; \text{after: } \text{assessed}(t) := \top$

(5) $\texttt{fit}_{\text{declarative}} \Rightarrow \forall k : \text{state}(\texttt{prep}_k) \perp \text{state}(\texttt{prep}_{j \neq k})$

(6) $\text{Evidence} \not\leq \text{Metrics}$ [`isinstance` distinguishable in host language]

(7) $\texttt{cv}(d).\{\text{train}, \text{valid}, \text{test}, \text{dev}\} \to \texttt{error}$

(8) $\texttt{evaluate}(m, d) \Rightarrow \text{tag}(d) \neq \text{test}$

The practical value of the conformance conditions is decidability: without a grammar, "does this workflow have structural leakage?" has no principled an-

swer. With it, structural correctness is checkable in constant time: eight primitives in the type DAG, four hard constraints, eight conformance conditions. Methodological adequacy (algorithm choice, metric selection, sample size) remains a judgment call. The test produces a binary verdict, not an opinion.

All eight conditions are verifiable from the API surface: `fit` rejects unregistered data with a `PartitionError` whose message directs the user to `split` first; `cv` returns a CVResult that raises on partition access. The closed-world limitation of prior versions (where untagged data passed silently) is closed.

5.3 Hierarchy of defaults

The grammar operates at two levels of explicitness:

Declarative (default): `ml.fit(s.train, "y")` handles algorithm selection, preprocessing, and cross-validation internally. The grammar's constraints are satisfied automatically. This is the entry point for newcomers and the default mode.

Explicit: Each step can be called independently for full control. `ml.prepare()`, `ml.fit()`, `ml.evaluate()` as separate operations. The grammar's constraints still apply: `assess` is still terminal, preparation is still per-fold. But the user sees each intermediate type.

The default mode does the correct thing without requiring the user to know why. The explicit mode makes the types visible for the user who wants to understand (or override) each step. This is the Wickham move: the correct workflow should be the path of least resistance, and the type system should be invisible until you need it.

6 Comparison with Existing Frameworks

The grammar claims to do something that, to my knowledge, no existing framework does. The table below tests this claim against the three closest alternatives on the four dimensions that matter for leakage prevention:

Framework	Per-fold prep	Terminal assess	Provenance	Reject at call time
sklearn	via Pipeline[p]	—	—	—
tidymodels	via recipes[p]	`last_fit()`[l]	—	—
ml (this grammar)	default	`assess()`[b]	content-addressed	**yes**

{[p]Inside Pipeline/workflow only; manual preprocessing outside leaks silently. [l]Encourages one-shot but does not guard against bypass. [b]On by default; `config(guards="off")` disables all guards. Operations outside the grammar's eight primitives (e.g., raw sklearn preprocessing) are not covered regardless. The difference is default posture: the grammar defaults to enforcement and requires an explicit exit; sklearn and tidymodels default to flexibility.}

The grammar leads on methodological enforcement (the four columns above) but trails on ecosystem breadth, maturity, and arbitrary code coverage. sklearn's 17-year ecosystem and tidymodels' integration with the R tidyverse are strengths the grammar does not replicate.

6.1 tidymodels

Kuhn and Silge (2022) is the closest existing framework. Its recipes enforce per-fold preprocessing when used inside workflows, preventing Class I leakage structurally. It deserves explicit credit as the most important prior work.

The grammar inherits tidymodels' insight that preprocessing must be per-fold, and extends it in one direction: terminal assessment. This is a design tradeoff, not a deficit in tidymodels: tidymodels optimizes for flexibility and ecosystem integration within R; the grammar optimizes for structural enforcement at the cost of ecosystem breadth. tidymodels types are implementation-level — recipe, parsnip model, workflow — and ensure that preprocessing steps are applied consistently. `last_fit()` is the closest existing analogue to the grammar's assess-once guard: it encourages a one-shot terminal evaluation on the test set after model selection is complete. But `last_fit()` is a recommended workflow function, not a guard; the user can bypass it and call `collect_metrics()` on test data repeatedly without raising an error. No type marks a model as assessed. The grammar closes that gap with a terminal type distinction that makes repeated test-set assessment a runtime failure. The two frameworks agree on what the correct workflow is; they differ on whether the incorrect one should be discouraged or rejected.

6.2 D3M and AutoML grammars

DARPA's D3M programme (2017–2021) developed ~280 typed primitives with automated pipeline search (Smith et al. 2020); Drori et al. (2021) extends D3M with context-free grammars over pipeline components. D3M's types serve AutoML search pruning, not methodological validity: a D3M-generated pipeline can still reuse the test set or cherry-pick seeds. Their grammars answer "what pipelines can I build?"; mine answers "what workflows are valid?"

6.3 NBLyzer and abstract interpretation

Drobnjaković, Subotić, and Urban (2024) applied abstract interpretation, a principled framework from programming language semantics, to ML leakage detection in Jupyter notebooks. Their system propagates partition-membership labels (Train, Test, Full, Unknown) through data-flow operations and flags any Test-labeled value flowing into a fit-class operation as a leakage error. Evaluated on 2,111 Kaggle notebooks, the approach achieved 93% precision. This is the closest existing work to a formal treatment of pipeline correctness.

The tradeoff is scope versus timing. Their tool examines completed code and catches violations the grammar cannot see — arbitrary Python, sklearn calls, operations outside the eight primitives. The grammar catches violations their tool cannot prevent — invalid operations blocked at call time, before they produce a number. Their tool is a post-mortem; the grammar is a guardrail. The two are complementary.

6.4 sklearn API design

Buitinck et al. (2013) formalized the fit/predict/transform protocol that sklearn standardized: duck-typed typestate where `predict` before `fit` raises `NotFittedError`. The grammar extends

this chain: sklearn enforces one transition (unfitted → fitted); the grammar enforces a multi-state chain (untagged → partitioned → fitted → assessed) covering the full assessment lifecycle.

6.5 mlr3

Binder et al. (2021) is the most formally specified existing framework: typed PipeOp inputs and outputs with graph-based composition. However, mlr3's types serve composition (how to build pipelines) rather than validity (what workflows are rejected based on epistemic grounds). mlr3 defines no rejection criterion; no workflow pattern is structurally invalid. The two are complementary: mlr3's engineering could serve as an implementation substrate for the grammar's constraints.

6.6 The novelty claim

The individual primitives (split, fit, evaluate, predict) are present in every ML framework. Their enumeration is not novel. The grammar contributes three things that, in combination, are:

1. **The terminal guard.** To my knowledge, no other ML framework enforces the evaluate/assess boundary at call time. Repeated test-set assessment is a runtime failure in a conforming implementation, not a style violation. K-metering on the validation partition makes optimization pressure visible before it accumulates.
2. **A specification.** The Appendix defines the type system, guards, and validity conditions precisely enough for independent reimplementation. To my knowledge, no existing framework publishes one.
3. **Empirical grounding.** Each constraint is connected to a measured effect size (Roth 2026b): peeking at $K = 10$ produces $d_z = 0.929$, seed inflation follows $\log(K)$ ($R^2 > 0.99$), and at practical sizes the noise is ∼90% of the measured effect. The constraints target what costs the most, not what sounds worst.

Enforcement mechanisms form a natural hierarchy by how hard they are to bypass: documentation < linting (Yang et al. 2022) < static analysis (Drobnjaković, Subotić, and Urban 2024) < call-time guards (this grammar) < dependent types. The grammar occupies the runtime-guard position, the strongest enforcement available in dynamically typed languages without external tooling.

7 Empirical Motivation

The grammar's formal properties are verifiable from the specification alone; a reader can check the Appendix without running a single experiment. But a grammar that rejects workflows nobody cares about is useless. A companion study (Roth 2026b) across 2,047 datasets measures how much each leakage type actually costs, so the constraints can be judged not just by logical necessity but by empirical severity:

Class	Mechanism	Measured effect	Grammar response
I	Estimation (normalize before split)	Negligible (ΔAUC < 0.005)	Constraint 2 (principled, costless)
II	Selection (peek, seed, screen)	Substantial at practical n	Constraints 1, 3, 4 + terminal assess

Class	Mechanism	Measured effect	Grammar response
III	Memorization (train on eval data)	Capacity-dependent	Constraint 4 (provenance gate)
IV	Boundary (partition $\neq$ deployment)	Domain-dependent (+0.023 where drift exists)	`split_temporal`, `split_group`

The four classes are defined by mechanism (Kaufman et al. 2012; Roth 2026b); the grammar targets the classes with large effects. Class I effects are negligible at any sample size, consistent with Bousquet and Elisseeff (2002)'s stability theory. Every selection mechanism decomposes into noise exploitation ($\sigma\sqrt{2\ln K}$, where $\sigma \sim 1/\sqrt{n}$) and genuine diversity (the true performance spread across the selection pool). At the corpus median ($n \approx 1{,}900$), approximately 90% of the measured Class II effect is real noise exploitation — selection bias that inflates reported scores. Under any violation of the i.i.d. assumption — temporal shift, group structure, spatial correlation — the inner-CV optimism becomes structural bias that does not decay with n, because more data from a biased sampling process does not correct the bias. Random cross-validation does not prevent this structural leakage — it censors the evidence by destroying the structure that would reveal it. The grammar's boundary-aware primitives (`split_temporal`, `split_group`, `cv_temporal`, `cv_group`) address the structural layer; the terminal assess gate addresses the selection layer on top. The measured leakage types are samples from an infinite category of provenance violations *within the grammar's scope*: the grammar prevents the category, not just the measured samples (this follows by construction from the guard definitions, not from empirical coverage). Contamination baked into the raw data before any primitive touches it — a feature that encodes the target, a generating process that shifted silently — is invisible to the provenance registry. Class III effects are capacity-dependent: decision trees memorize more than random forests. Full effect sizes, decomposition analysis, and confidence intervals are reported in the companion study (Roth 2026b).

The design is within-subject counterfactual: a clean workflow and a leaky workflow run on the same data, the same folds, the same seed. Only the leakage perturbation differs. Both outcomes are observed for every dataset, so each ΔAUC is an individual treatment effect, not an estimate. All AUC values are computed from out-of-fold predictions. The 2,047 datasets were split at the *dataset level* into discovery ($k = 1{,}007$) and confirmation ($k = 1{,}040$) halves by deterministic MD5 hash, fixed before analysis. Every finding replicates on the independent confirmation half with zero failures. Three grammar predictions were tested before observing results: two confirmed, one falsified (Roth 2026b).

8 Discussion

8.1 What the grammar does not do

A grammar that prevents everything prevents nothing useful: it would reject valid workflows alongside invalid ones. The grammar's scope is deliberately narrow: structural errors where the evaluation pipeline is compromised. Semantic errors (choosing a wrong algorithm, a misleading metric, or an inappropriate model for the domain) are the practitioner's judgment, not the grammar's jurisdiction:

Valid in the grammar but poor in practice	Why the grammar allows it
Logistic regression on 1M rows when XGBoost dominates	Algorithm choice is not structural
Accuracy on a 99/1 imbalanced dataset	Metric selection is not structural
k-fold CV on time-series data	Temporal awareness requires domain knowledge
Normalizing binary features	Feature-level decisions are semantic

This is the ML analogue of Chomsky's distinction between syntactic and semantic well-formedness. "Colorless green ideas sleep furiously" is grammatically valid but semantically nonsense (Chomsky 1957). A workflow that passes all type checks but produces a poor model for semantic reasons is the ML analogue. Grammatical compliance is necessary for structural validity but not sufficient for scientific quality. A diagnostics layer (`check`, `drift`, `profile`, `enough`) addresses semantic quality as a separate concern.

Structural validity does not imply numerical reproducibility. `fit(data, seed=42)` and `fit(data, seed=43)` are grammatically identical; they may produce models with different metrics and different conclusions. The grammar prevents structural leakage within its eight primitives, not non-reproducibility — both are correctness concerns, but only one is structural. Reproducibility needs fixed seeds, environment versioning, and hardware determinism; those are infrastructure problems, not grammar problems.

The primitive set is operation-centric: all eight primitives are verbs. This is a design choice, not an oversight. Validity constraints are enforced at operations, not at values: a DataFrame is neither valid nor invalid in isolation; it becomes invalid when passed to `fit` without a partition tag. The types (DataFrame, Partition, Model, Metrics) are defined formally in the Appendix and are co-equal in the specification; they appear secondary in the prose because the actionable grammar lives on the verbs, not the values flowing between them.

8.2 Small N

Below approximately $N = 30$, the grammar's three-way split becomes statistically vacuous. The split does not disappear; it contracts:

N	Grammar mode	Recommendation
< 10	Does not apply	Exact tests, Bayesian, case studies
10–29	LOO-CV, one model	Report massive uncertainty
30–199	k-fold CV, 1–3 models	No screening, no stacking
200–999	Full grammar feasible	Screen, tune, holdout
≥ 1000	Grammar in its element	All strategies available

The deeper constraint is not raw N but the ratio of events to predictors: logistic regression needs approximately 10–20 events per predictor (see Riley et al. 2019 for modern guidance), while tree ensembles need 3,000+ (van der Ploeg, Austin, and Steyerberg 2014). The grammar's real contribution at small N is discipline: separate what you learn on from what you judge on. That discipline applies at every N. The mechanism of separation requires a minimum investment of observations.

8.3 Domain scope

The grammar covers tabular supervised learning (Section 4). Both `evaluate` and `assess` assume a stationary data-generating process; under concept drift, the type guarantees hold but `Evidence` no longer measures generalization to the current environment. Extending the grammar to other paradigms requires analogous empirical baselines first.

8.4 External validation

External validation is the missing piece and the paper's most important open question. This paper is a specification paper with reference implementations: the type system, guards, and conformance conditions are verifiable from the specification alone. Whether the grammar reduces leakage errors in practice is a separate empirical question that the specification does not answer. Wickham had ggplot2 users before the grammar paper; here the sequence is reversed. But when pipelines are increasingly generated rather than written, the grammar's value does not depend on human adoption — it depends on whether the APIs that LLMs call enforce correctness the models themselves cannot. A between-subjects experiment comparing leakage rates in student code written with the grammar versus sklearn, scored by an automated checker blind to condition, would test the central claim directly. The relevant effect size is not the leakage magnitude ($d_z = 0.929$ for Class II) but the difference in leakage *rates* between conditions — a behavioral quantity that is unknown until the study is run. If the grammar eliminates leakage in the treatment arm entirely (the structural guarantee predicts this), the effect size depends on the baseline leakage rate in the control arm. At a conservative $d = 0.8$, a two-group design with $n = 25$ per arm achieves 80% power at $\alpha = 0.05$; the study is feasible at classroom scale. If the grammar does not measurably reduce leakage in human-written code, the type system is correct but practically inert. That outcome is possible and would be informative.

A partial answer comes from LLM-generated code. A companion study in preparation (Roth 2026a) uses LLMs as a proxy for code authors, prompted at scale with controlled dose-response designs that would be infeasible with human subjects. Preliminary results suggest that when asked to build an ML pipeline, the majority of models default to the shortest path — and the shortest path leaks. If confirmed, this would mean that structural enforcement at the API level is the only guarantee that scales when pipelines are generated at machine speed. The human between-subjects design remains the definitive test.

8.5 Enforcement gaps

All implementations use gates, not funnels: `fit`, `evaluate`, and `assess` reject data that has not passed through `split`. The tradeoff: `ml.fit(df, "y")` without a prior `split` call no longer works by default; `config(guards="off")` provides an explicit, auditable exit analogous to Rust's `unsafe` blocks.

The provenance system uses *content-addressed partition identity*: `split` computes a deterministic fingerprint of each output partition and registers it in a session-scoped provenance registry. Guards query the registry by content fingerprint, not metadata attribute, so provenance survives host-language operations that strip attributes (`merge`, `concat`, `groupby` in pandas; `dplyr`, `rbind` in R). This closed the most significant structural gap in prior versions — *tag erasure* — where typed metadata was silently discarded by untyped operations in the surrounding environment.

Three known gaps remain. (1) The registry is session-scoped; a kernel restart resets the assess-once counter. Persisting the registry as signed breadcrumb files would extend the guarantee across sessions. (2) DataFrames with unhashable column types bypass fingerprinting silently — the one case where the guard architecture has a genuine blind spot rather than a conscious tradeoff. (3) The assess-once constraint is per holdout, not per model: a practitioner who trains 5 models and assesses each on the same test set has performed model selection on test data. The design accepts this tradeoff — model selection belongs in the evaluate zone, and the provenance registry marks the holdout as spent after the first assessment regardless of model identity.

A grammar is a specification; any implementation is an approximation of it. Early SQL implementations violated Codd's relational model routinely; the specification remained correct, and implementations improved over decades.

8.6 Production and deployment

Production retraining follows the same `split` → `cv` → `fit` → `evaluate` → `assess` workflow, so the existing guards apply without modification. The reference implementations include a `drift` diagnostic verb for monitoring distributional shift without requiring new primitives.

8.7 Optimization leakage

The grammar closes data leakage but does not fully close optimization leakage: a practitioner who runs `screen` with 11 algorithms, `tune` with 500 configurations, and then calls `assess` has made hundreds of model selection decisions on the validation set. The expected maximum of K noisy evaluations exceeds the population mean by $\sigma\sqrt{2 \ln K}$ (Gumbel 1958; Cawley and Talbot 2010).

The grammar addresses this through progressive access control. The three partitions operate at different speeds:

1. **Train (unrestricted).** Iteration within the training partition is unlimited and honest: cross-validation folds are ephemeral, created and consumed within the inner loop.
2. **Valid (metered).** Each `evaluate` call increments a per-partition counter K in the provenance registry. The closest prior art is Blum and Hardt (2015), who metered holdout access in ML competitions; the present mechanism applies the same principle inside the practitioner's workflow.
3. **Test (terminal).** One assessment per holdout. Enforced by the assess-once constraint.

This progressive slowdown — free on train, metered on valid, terminal on test — mirrors the natural structure of disciplined research: explore freely, steer occasionally, commit once. The assess-once constraint is the zero-reuse limit of Dwork et al. (2015)'s adaptive framework ($\epsilon = 0$): rather than permitting controlled repeated access with formal privacy guarantees, it permits no reuse at all. This trades the reusable holdout's flexibility for enforceability in a static type system — engineering discipline rather than a privacy guarantee.

9 Conclusion

The evaluate/assess boundary is not just a textbook rule to follow; it is a constraint this grammar enforces. Its type system remembers the rules so you don't have to. Time currently spent on unsafe exploration (fitting on test data, leaking preprocessing across folds, re-assessing holdouts until results look favorable) is forced into the evaluate zone, where iteration is unlimited but metered, and therefore honest.

Eight primitives, four constraints, two reference implementations (Python, R), and a companion study across 2,047 datasets (Roth 2026b): the grammar is internally consistent and the implementations satisfy the conformance conditions. At practical dataset sizes, the leakage classes the grammar rejects produce substantial effects — approximately 90% noise exploitation ($\sigma\sqrt{2 \ln K}$) that is substantial, empirically grounded, and preventable at call time by rejection. Under any violation of the i.i.d. assumption — the regime of every real deployment — the selection bias becomes irreducible because past evaluation is a biased estimator of future performance, and random cross-validation censors the evidence by destroying the structure that would reveal it. The grammar addresses both layers: boundary-aware splitting for the structural contamination, K-metering for making selection pressure visible, and terminal assess as the final safety net for assumption violations and implementation bugs that no earlier layer caught. Three directional predictions recorded before observing results: two confirmed, one falsified. The grammar is falsifiable and has survived its initial tests.

The type DAG and four constraints are frozen at v1.0. Whether the grammar reduces leakage errors in practice remains the central empirical question (Section 8.4). Preliminary results from a companion study (Roth 2026a) suggest that it eliminates a class of error in LLM-generated code that persists even under explicit natural-language guidance. As pipelines are increasingly generated rather than written, the grammar's role shifts from constraining the human to constraining the generator.

Leakage and bias are distinct failures, but they share a root cause: pipeline stages that should be isolated are not (Roth 2022). A model whose training process is contaminated by test data cannot be trusted to measure bias either. The grammar does not detect bias, but it closes the methodological precondition: honest measurement requires honest evaluation, and honest assessment is what the type system enforces.

A researcher who fits her imputer on the full dataset before splitting is not cutting corners; she is using software that treats the split as an afterthought. A social scientist who selects predictors on the pooled sample before cross-validating is not unfamiliar with overfitting; his statistics package presented variable selection and validation as independent steps. An economist who reports in-sample R^2 is not ignorant of holdout methodology; the holdout was two menu options deeper buried than the default workflow. An automated retraining pipeline that refits monthly on all available data, including the month it will immediately score, is not malfunctioning; it is fulfilling its update schedule. A language model asked to generate a pipeline will select the best out of 100 model families on the test set — because that is the modal pattern in its training distribution, and no gradient pointed away from it. The 648 papers with leakage errors are not anomalies; they are the expected output of a field whose methodological knowledge outpaced its tooling.

The grammar does not require the field to learn something new. It requires the tools to enforce what the field already knows. The training boundary should be a first-class type, and leakage should be a violation, not a matter of professional discipline. Professional discipline varies across career stages, institutions, and time zones. Type systems do not. As pipelines are increasingly generated at machine speed — by AutoML systems, LLM code generators, and automated retraining schedules — the structural guarantee becomes the only guarantee that scales. Investment allocation, credit scoring, medical diagnosis, recidivism prediction: when the pipeline that produces the number runs faster than any methodology review can follow, the question is not whether the practitioner knows the rules. The question is whether the tool enforces them at call time.

Appendix: Formal Specification

9.1 Type system

The grammar is a typed specification over nine types. Each primitive has a type signature, a guard (precondition checked at call time), and an effect (state mutation after call). Three types are terminal (their values feed no further primitive):

Type	Structure	Terminal
DataFrame	Tabular data with named columns and `partition_tag: {None, train, valid, test, dev}`. Freshly loaded data carries tag `None` (untagged). `split` assigns tags `train`, `valid`, `test` to the output partitions. The `dev` tag is assigned to the union of train and valid (`s.dev`).	No
Partition	`{train, valid, test, dev: DataFrame}` where `dev = train  valid`	No
CVResult	`{folds: [(train_idx, valid_idx)]^k, k: N, target: str}`. Produced by `cv` from a training or dev DataFrame; blocks `.train`, `.valid`, `.test`, `.dev` access. Test partition stays on the originating Partition.	No
PreparedData	`{data: DataFrame_numeric, state: Transformer, target: str, task: {clf, reg}}`	No
Model	`{algorithm: str, task: {clf, reg}, fitted: bool}`‡	No
Predictions	One-column numeric DataFrame	No†
Metrics	`str → float`	Yes
Evidence	Sealed named type wrapping `str → float`; must be a class distinct from Metrics in the host language, not implicitly substitutable for Metrics under structural or duck typing. No primitive accepts Evidence as input.	Yes
Explanation	`str → float` (importances or partial dependences)	Yes

† Predictions is not terminal in stacking strategies: the `stack` strategy reshapes out-of-fold predictions into a new DataFrame and passes it to a second `fit` call (the meta-learner). In the grammar's 8-primitive DAG, no primitive accepts Predictions directly as a typed input; the reshape is performed by the strategy orchestration, not a primitive.

‡ The implementations additionally track `assessed` as an integer counter on the model (`_assess_count` in Python, `assess_count` in R) for fast-path diagnostics. The structurally load-bearing enforcement is per-holdout: the provenance registry tracks which test partitions have been assessed, and the guard rejects any subsequent call on the same partition regardless of model identity.

Two embedded types appear in the type table above but are not primitive types:

- `DataFrame_numeric`: a `DataFrame` where every column is numeric; a structural subtype of `DataFrame` (all rows of `DataFrame_numeric` are valid `DataFrame` values but not vice versa). Produced by `prepare`; consumed by model algorithms.
- `Transformer`: an interface type satisfying `state.transform(X: DataFrame) → DataFrame_numeric`. It encodes the fit-time preprocessing state so that validation data can be transformed consistently without re-fitting. Not a free-standing type; it is a field type embedded in `PreparedData`.

- `partition_tag`: an enum attribute `{None, train, valid, test, dev}` on every DataFrame. Not a separate type; it is an attribute of the `DataFrame` type. Freshly loaded data carries `None`; `split` assigns `train`, `valid`, `test`; accessing `s.dev` (train ∪ valid) assigns `dev`.

9.2 Primitive operations

Each primitive σ has a type signature, a guard $G(\sigma)$ (precondition checked at call time), and an effect $E(\sigma)$ (state mutation after call). **Registered** means the DataFrame's content fingerprint appears in the session-scoped provenance registry with a valid `partition_tag` assigned by a prior `split` call:

Primitive	Input	Output	Guard G	Effect E
split	DataFrame, params	Partition	—	assigns `partition_tag in {train, valid, test, dev}` to output partitions; registers content fingerprints of all partitions (including `dev = train valid`) in the provenance registry with their tags and split lineage
cv	DataFrame, params	CVResult	`data` registered by `split`; `data.partition_tag in {train, dev}`	blocks `.train`, `.valid`, `.test`, `.dev` on result; test stays on originating Partition
prepare	DataFrame, target	PreparedData	`data` registered by `split`; `data.partition_tag in {train, valid, dev}`	fits Transformer on data; stores in `PreparedData.state`
fit	DataFrame | CVResult | PreparedData, target	Model	`data` registered by `split`; if DataFrame, `data.partition_tag in {train, valid, dev}`; if CVResult, produced by `cv`; if PreparedData, inherits provenance from its source DataFrame	`model.fitted ← true`; prepare applied per fold (skipped if input is PreparedData)
predict	Model, DataFrame	Predictions	`model.fitted = true`	—

Primitive	Input	Output	Guard G	Effect E
evaluate	Model, DataFrame	Metrics	`model.fitted = true`; `data` registered by `split`; `data.partition_tag != test`	—
explain	Model [, DataFrame]	Explanation	`model.fitted = true`	—
assess	Model, DataFrame	Evidence	`data` registered by `split`; `data.partition_tag = test`; test partition not previously assessed in provenance registry	mark test partition as assessed in registry; `model.assessed ← true` (diagnostic)

9.3 When is a workflow valid?

A workflow (a sequence of primitive calls) is **valid** if:

1. **Types connect:** each primitive's output type matches the next primitive's expected input type.
2. **Guards pass:** each primitive's precondition holds at call time, given all prior effects.
3. **Effects apply in order:** state mutations (e.g., test partition marked as assessed in registry) accumulate sequentially.

A workflow is **invalid** if any step fails a type check or a guard. The four hard constraints are guards in this sense: not rules a user must remember, but checks that reject invalid workflows at call time.

Note on evaluate guard. The guard on `evaluate` rejects test-tagged data; it accepts train and valid partitions. Evaluating on training data is legitimate (train-vs-valid score comparison for overfitting diagnosis). The guard prevents test data from entering the iterate cycle: if `evaluate` accepted test data, the practitioner could iterate on test-set feedback, which is structurally equivalent to training on the test set. The `assess` primitive is the only path for test data. This is a design decision: `evaluate` is the practice exam (repeatable, on validation data); `assess` is the final exam (terminal, on test data).

Note on implementation separation. `assess` and `evaluate` are independent primitives with no call relationship. Both delegate to a shared private scoring function for prediction and metrics computation, but neither calls the other. `assess` enforces the test-tag requirement and the once-per-holdout constraint; `evaluate` enforces the test-data rejection guard. Strategy verbs (`compare`, `validate`, `screen`, `shelf`, `tune`) that need metrics on arbitrary partitions call the same shared scoring function directly, bypassing both `evaluate`'s and `assess`'s guards and enforcing their own partition constraints. The type DAG correctly shows `assess` and `evaluate` as independent primitives.

Note on branching strategies. This definition covers linear workflows. Branching strategies (screen, tune, stack) involve parallel or iterated applications of the same primitives; their validity follows from applying the same type signatures and guards to each branch independently.

Note on domain specializations. `split` admits domain specializations (`split_temporal`, `split_group`) that share its type signature but carry additional guards specific to a scientific domain. This is a representational mechanism, not a primitive extension: the guard set

G(split) is parameterized by domain, but the type signature and position in the DAG are invariant. Time series forecasting, for example, would add temporal guards to `split` without requiring new primitives.

Scope. The guarantees apply to programs that compose the grammar's 8 primitives. Operations outside these eight (arbitrary DataFrame transformations, external feature selection, direct attribute mutation) are outside the grammar's scope. The grammar enforces its own API boundary; it cannot enforce what happens in surrounding code.

9.4 Invalid workflows as guard failures

Workflow	Failing condition	Mechanism
assess(m) → assess(m)	G(assess): test partition not previously assessed	First call marks test fingerprint as assessed in registry; second call on same holdout fails
assess(m_1, test) → assess(m_2, test)	G(assess): test partition not previously assessed	First call marks test fingerprint as assessed in registry; second call on same holdout fails regardless of model
prepare(X_{all}) → split → fit	G(prepare): data not registered by `split`	`prepare` rejects unregistered data (`partition_tag = None`); failure occurs at `prepare`, before `split` is called
select_features(X, y) → split → fit	`fit` requires split provenance	Feature selection belongs inside `prepare`, per fold (§ Feature engineering); unregistered data rejected
evaluate without prior fit	Type continuity	No Model in scope; `out(split) = Partition != in(evaluate) = Model`
fit(s.test)	G(fit): `data.partition_tag not in {train, valid, dev}`	Test tag is `test`, not in `{train, valid, dev}`; guard fails
evaluate(m, s.test)	G(evaluate): `data.partition_tag != test`	Test tag triggers guard; test data is reserved for `assess`

API maturity. The implementations are versioned at 1.x and follow semantic versioning. The 8 kernel primitives and 4 hard constraints are stable; the strategy-layer verbs and their signatures may change before 2.0. The grammar specification (Appendix A) is the stable contract; individual implementations should be treated as reference rather than frozen.

Code and Data Availability

The Python implementation is available as `mlw` on PyPI (`pip install mlw`, imported as `import ml`). The R implementation is available as `ml` on CRAN and on GitHub (`epagogy/ml`). Source code for both implementations: github.com/epagogy/ml. The specification (Appendix A) is sufficient for independent reimplementation.

Conflict of Interest

The author develops and distributes the Python and R implementations described in this paper. The grammar specification is intended to be implementable independently of them.

Acknowledgments

This work was conducted independently and received no external funding. I thank the colleagues and peers who provided critical feedback during this process; they will be acknowledged individually upon journal submission, if they choose to be named. This is ongoing work; feedback is welcome at simon@epagogy.ai.

Disclosure

Large language models (Claude, Anthropic) were used as principal writing, analysis, and implementation tools during the preparation of this manuscript. All scientific claims, experimental designs, empirical results, and theoretical contributions are my own. I take full responsibility for the content.

References

Ballarin, Giovanni, Petros Dellaportas, Lyudmila Grigoryeva, Marcel Hirt, Sophie van Huellen, and Juan-Pablo Ortega. 2024. "Reservoir Computing for Macroeconomic Forecasting with Mixed-Frequency Data." *International Journal of Forecasting* 40 (3): 1206–37. https://doi.org/10.1016/j.ijforecast.2023.10.009.

Bates, Stephen, Trevor Hastie, and Robert Tibshirani. 2024. "Cross-Validation: What Does It Estimate and How Well Does It Do It?" *Journal of the American Statistical Association* 119 (546): 1434–45. https://doi.org/10.1080/01621459.2023.2197686.

Binder, Martin, Florian Pfisterer, Michel Lang, Lona Schneider, Lars Kotthoff, and Bernd Bischl. 2021. "Mlr3pipelines: Flexible Machine Learning Pipelines in R." *Journal of Machine Learning Research* 22 (184): 1–7. https://jmlr.org/papers/v22/21-0206.html.

Blum, Avrim, and Moritz Hardt. 2015. "The Ladder: A Reliable Leaderboard for Machine Learning Competitions." In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, 1006–14. https://arxiv.org/abs/1502.04585.

Bousquet, Olivier, and André Elisseeff. 2002. "Stability and Generalization." *Journal of Machine Learning Research* 2: 499–526. https://jmlr.org/papers/v2/bousquet02a.html.

Buitinck, Lars, Gilles Louppe, Mathieu Blondel, Fabian Pedregosa, Andreas Mueller, Olivier Grisel, Vlad Niculae, et al. 2013. "API Design for Machine Learning Software: Experiences from the Scikit-Learn Project." In *ECML PKDD Workshop: Languages for Data Mining and Machine Learning*, 108–22. https://arxiv.org/abs/1309.0238.

Cawley, Gavin C., and Nicola L. C. Talbot. 2010. "On over-Fitting in Model Selection and Subsequent Selection Bias in Performance Evaluation." *Journal of Machine Learning Research* 11: 2079–2107. https://jmlr.org/papers/v11/cawley10a.html.

Chomsky, Noam. 1957. *Syntactic Structures*. The Hague: Mouton.

Codd, Edgar F. 1970. "A Relational Model of Data for Large Shared Data Banks." *Communications of the ACM* 13 (6): 377–87. https://doi.org/10.1145/362384.362685.

Drobnjaković, Filip, Pavle Subotić, and Caterina Urban. 2024. "Abstract Interpretation for Data Leakage Detection in Machine Learning Pipelines." In *Theoretical Aspects of Software Engineering (TASE 2024)*. https://doi.org/10.1007/978-3-031-64626-3_7.

Drori, Iddo, Yamuna Krishnamurthy, Remi Rampin, Raoni de Paula Lourenco, Jorge Piazentin Ono, Kyunghyun Cho, Claudio Silva, and Juliana Freire. 2021. "AlphaD3M: Machine Learning Pipeline Synthesis and AutoML."

Dwork, Cynthia, Vitaly Feldman, Moritz Hardt, Toniann Pitassi, Omer Reingold, and Aaron Roth. 2015. "The Reusable Holdout: Preserving Validity in Adaptive Data Analysis." *Science* 349 (6248): 636–38. https://www.science.org/doi/10.1126/science.aaa9375.

Gumbel, Emil Julius. 1958. *Statistics of Extremes*. New York: Columbia University Press.

Hastie, Trevor, Robert Tibshirani, and Jerome Friedman. 2009. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2nd ed. New York: Springer. https://hastie.su.domains/ElemStatLearn/.

Kapoor, Sayash, Emily M. Cantrell, Kenny Peng, Thanh Hien Pham, Christopher A. Bail, Odd Erik Gundersen, Jake M. Hofman, et al. 2024. "REFORMS: Consensus-Based Recommendations for Machine-Learning-Based Science." *Science Advances* 10 (18): eadk3452. https://doi.org/10.1126/sciadv.adk3452.

Kapoor, Sayash, and Arvind Narayanan. 2025. "Leakage and the Reproducibility Crisis in ML-Based Science — Living Survey." https://reproducible.cs.princeton.edu.

Kaufman, Shachar, Saharon Rosset, Claudia Perlich, and Ori Stitelman. 2012. "Leakage in Data Mining: Formulation, Detection, and Avoidance." *ACM Transactions on Knowledge Discovery from Data* 6 (4): 1–21. https://doi.org/10.1145/2382577.2382579.

Kuhn, Max, and Julia Silge. 2022. *Tidy Modeling with R*. Sebastopol, CA: O'Reilly Media. https://www.tmwr.org.

Myers, Andrew C. 1999. "JFlow: Practical Mostly-Static Information Flow Control." In *Proceedings of the 26th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, 228–41. https://doi.org/10.1145/292540.292561.

Pedregosa, Fabian, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, et al. 2011. "Scikit-Learn: Machine Learning in Python." *Journal of Machine Learning Research* 12: 2825–30. https://jmlr.org/papers/v12/pedregosa11a.html.

Riley, Richard D., Kym I. E. Snell, Joie Ensor, Danielle L. Burke, Frank E. Harrell, Karel G. M. Moons, and Gary S. Collins. 2019. "Minimum Sample Size for Developing a Multivariable Prediction Model: Part I — Continuous Outcomes." *Statistics in Medicine* 38 (7): 1262–75. https://doi.org/10.1002/sim.7993.

Rosenblatt, Matthew, Link Tejavibulya, Rongtao Jiang, Stephanie Noble, and Dustin Scheinost. 2024. "Data Leakage Inflates Prediction Performance in Connectome-Based Machine Learning Models." *Nature Communications* 15: 1829. https://doi.org/10.1038/s41467-024-46150-w.

Roth, Simon. 2022. "Biased Machines in the Realm of Politics." Dr.\ rer.\ soc.\ dissertation, Universität Konstanz. https://kops.uni-konstanz.de/handle/123456789/59732.

———. 2026a. "The Shortest Path Leaks: How LLM-Generated ML Pipelines Inherit and Amplify Data Leakage."

———. 2026b. "Which Leakage Types Matter? A Quantitative Landscape Across 2,047 Benchmark Datasets." https://doi.org/10.5281/zenodo.19406148.

Scriven, Michael. 1967. "The Methodology of Evaluation." In *Perspectives of Curriculum Evaluation*, edited by Ralph W. Tyler, Robert M. Gagné, and Michael Scriven, 39–83. Chicago: Rand McNally.

Smith, Micah J., Carles Sala, James Max Kanter, and Kalyan Veeramachaneni. 2020. "An Interactive Pipeline for Cross-Domain AutoML Search." In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, 2735–38. https://doi.org/10.1145/3318464.3384693.

Stone, Mervyn. 1974. "Cross-Validatory Choice and Assessment of Statistical Predictions." *Journal of the Royal Statistical Society: Series B (Methodological)* 36 (2): 111–33. https://doi.org/10.1111/j.2517-6161.1974.tb00994.x.

Strom, Robert E., and Shaula Yemini. 1986. "Typestate: A Programming Language Concept for Enhancing Software Reliability." *IEEE Transactions on Software Engineering* SE-12 (1): 157–71. https://doi.org/10.1109/TSE.1986.6312929.

Tampu, Iulian Emil, Anders Eklund, and Neda Haj-Hosseini. 2022. "Inflation of Test Accuracy Due to Data Leakage in Deep Learning-Based Classification of OCT Images." *Scientific Data* 9: 580. https://doi.org/10.1038/s41597-022-01618-6.

van de Mortel, Thomas F., and Guido A. van Wingen. 2025. "Data Leakage in Machine Learning Studies Creep into Meta-Analytic Estimates." *Molecular Psychiatry*. https://doi.org/10.1038/s41380-025-03336-y.

van der Ploeg, Tjeerd, Peter C. Austin, and Ewout W. Steyerberg. 2014. "Modern Modelling Techniques Are Data Hungry: A Simulation Study for Predicting Dichotomous Endpoints." *BMC Medical Research Methodology* 14: 137. https://doi.org/10.1186/1471-2288-14-137.

Wickham, Hadley. 2010. "A Layered Grammar of Graphics." *Journal of Computational and Graphical Statistics* 19 (1): 3–28. https://doi.org/10.1198/jcgs.2009.07098.

Wilkinson, Leland. 1999. *The Grammar of Graphics*. Statistics and Computing. New York: Springer.

Yang, Chenyang, Rachel A. Brower-Sinning, Grace A. Lewis, and Christian Kaestner. 2022. "Data Leakage in Notebooks: Static Detection and Better Processes." In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, 1–12. https://doi.org/10.1145/3551349.3556918.